

LIBCom: Enabling Low-Cost Intra-Bank Communication for Processing-in-Memory

Abstract—Processing-in-Memory (PIM) architectures aim to address the memory wall challenge by placing computation as close as possible to the data. In many PIM designs, lightweight processing elements (PEs) are integrated within or near SRAM subarrays to exploit locality. However, once computations produce intermediate results, efficient communication between subarrays becomes critical. Conventional SRAM architectures lack such intra-memory communication support, since data must first exit the memory before being reinserted, causing high latency and energy overhead. To address this bottleneck, we propose LIBCom, a switch-based SRAM interconnect that enables direct, low-cost inter-subarray data movement with minimal modifications to traditional SRAM designs.

Index Terms—Processing-in-memory, SRAM, subarray, communication, interconnect

I. INTRODUCTION

The memory wall challenge continues to worsen as the gap between processor performance and memory access latency grows exponentially [1]. Traditional memory hierarchies, despite decades of optimization through caching and prefetching, cannot adequately address the data movement bottlenecks faced by modern computing systems. The diversification of applications across data centers and specialized domains has further exacerbated this challenge [2], as diverse workloads demand fundamentally different data access patterns and communication requirements. To address these limitations, PIM architectures have emerged as a promising solution by placing computation directly within or adjacent to memory arrays [3], [4]. By moving processing closer to data, PIM systems can exploit data locality and reduce the energy and latency penalties associated with traditional processor-memory data transfers [5]. However, as PIM architectures perform computations within memory, they generate intermediate results and processed data that must be efficiently communicated between different memory regions for subsequent processing stages. This requirement for efficient intra-memory communication has become the new bottleneck in PIM systems [6].

Many deep learning accelerators partition computations across multiple processing elements (PEs), each with dedicated SRAM buffers, requiring frequent data exchanges between these local memories [7]–[9]. Architectures like Tangram [10] demonstrate the need for dynamic data rotation between processing buffers to maximize effective memory capacity, while large-scale neural networks with attention mechanisms and complex data dependencies require extensive data movement within the memory subsystem. When lightweight PEs are placed directly within or adjacent to SRAM subarrays [11]–[13], these distributed PEs must efficiently communicate with

neighboring subarrays for subsequent processing stages. Current SRAM architectures fundamentally lack support for such direct inter-subarray communication, forcing data to traverse the entire memory hierarchy even when source and destination reside within the same physical memory chip.

The root cause of this inefficiency lies in the traditional H-tree interconnect design in SRAM, which only supports communication between individual subarrays and the central memory I/O interface. Any data movement between subarrays requires a costly round-trip through the H-tree hierarchy: data must first travel from the source subarray to the memory controller, be temporarily stored in external buffers, and then be written back through the H-tree to the destination subarray. This circuitous path has been shown to consume up to 80% of total energy in large SRAM structures [14], representing a fundamental bottleneck for PIM applications that rely on frequent inter-subarray data exchanges.

Existing solutions for intra-memory communication operate at coarse granularities that are inadequate for fine-grained subarray-level communication. Approaches like Row-Clone [15] enable fast bulk data copy within DRAM but cannot address communication needs between SRAM subarrays. Similarly, network-based solutions for DRAM such as DIMM-Link [16], ABC-DIMM [17], and PIMNet [6] provide inter-rank to inter-bank communication capabilities, but these coarse-grained solutions are designed for more general-purpose PEs rather than the fine-grained processing capabilities integrated within or adjacent to individual SRAM subarrays. Furthermore, many analog PIM solutions that perform computation using charge-sharing or current-mode operations critically require inter-subarray networks to aggregate partial results and enable complex multi-subarray computations [12], [13], yet existing solutions cannot provide the necessary connectivity.

To address these limitations, we propose LIBCom, a low-cost switch-based interconnect that enables direct communication between SRAM subarrays with minimal modifications to existing memory designs. Unlike previous approaches that require significant changes to memory architecture or impose restrictions on data movement patterns, LIBCom introduces a simple yet effective network that allows arbitrary inter-subarray data transfers at fine granularity. Our approach maintains compatibility with existing SRAM designs while providing the communication infrastructure necessary for efficient PIM operations with lightweight PEs. Key contributions of this work include:

- We present a novel low-cost switch-based interconnect

design that enables direct inter-subarray communication with minimal modifications to existing SRAM designs.

- The design includes a lightweight routing mechanism at the peripheral circuitry that supports arbitrary source-destination pairs within the same memory bank while maintaining minimal overhead.
- Comprehensive evaluations demonstrate significant performance improvements for diverse workloads compared to traditional SRAM architectures.
- Detailed analysis reveals the energy and area trade-offs, showing that our approach achieves efficient inter-subarray communication with acceptable overhead.

The remainder of this paper is organized as follows. Section II provides background on traditional SRAM H-tree interconnects and reviews related works. Section III presents the LIBCom architecture, detailing the switch-based interconnect design and routing mechanisms. Section IV describes our evaluation methodology, including both circuit-level and system-level analysis approaches. Section V presents comprehensive experimental results comparing LIBCom against traditional SRAM architectures and further discussions beyond the evaluation results. Finally, Section VI concludes the paper and discusses future research directions.

II. BACKGROUND AND RELATED WORK

A. Traditional SRAM Architecture and the H-Tree Bottleneck

Traditional SRAM memories are organized hierarchically into banks, with each bank divided into smaller subarrays as shown in Fig. 1. Address decoding occurs in stages, where the most significant bits identify the target bank and subsequent bits determine the specific subarray. This hierarchical organization enables scalable design but creates a fundamental limitation: all data communication must flow through the central H-tree interconnect that connects each subarray to the bank I/O interface.



Fig. 1. Traditional SRAM bank architecture with 4 subarrays

While this architecture proves highly efficient for traditional processor-memory interactions, it becomes a critical bottleneck for PIM applications requiring inter-subarray communication. Any data transfer between subarrays necessitates a costly round-trip through the H-tree: data must travel from the source subarray to the I/O interface, be temporarily buffered externally, and then traverse back through the H-tree to reach the destination subarray. This inefficient path consumes substantial energy and introduces significant latency, particularly problematic for PIM systems where frequent inter-subarray data exchanges are essential for computational efficiency.

B. Limitations of Existing Inter-Memory Communication Solutions

Current approaches to accelerating intra-memory data movement fall into several categories, each with significant

limitations for subarray-level communication in PIM systems. **DRAM-specific solutions** such as RowClone [15], ABC-DIMM [17], and DIMM-Link [16] provide efficient bulk data copy and inter-rank communication capabilities within DRAM systems. However, these approaches cannot address the communication requirements of SRAM-based PIM architectures where PEs are integrated within or adjacent to individual subarrays. The fundamental differences in DRAM and SRAM technologies, along with their distinct organizational structures, prevent direct application of these techniques to SRAM systems. More fine-grained approaches like PIMNet [6] enable communication between PEs in PIM systems but still operate between DRAM banks. These solutions address inter-chip or inter-bank communication but cannot provide the subarray-level connectivity required for efficient analog PIM operations that rely on charge-sharing or current-mode computations across multiple subarrays.

SRAM-specific solutions also exhibit various fundamental limitations that prevent their adoption for flexible inter-subarray communication. Bitline-constrained approaches [12], [18], [19] enable data movement only between locations connected to the same bitlines, severely limiting the scope of possible data transfers and preventing communication between different subarrays. Subarray partitioning methods [20], [21] require substantial architectural modifications that split subarrays into predetermined regions, offering limited flexibility and significantly increasing design complexity. Similarly, indirection-based techniques [22] delay actual data movement through pointer manipulation, which is inadequate for PIM applications requiring immediate access to relocated data for subsequent computations. Near-memory processing solutions [14] add processing units adjacent to memory but still require data to traverse the H-tree infrastructure, failing to eliminate the fundamental energy bottleneck of inter-subarray communication.

None of these existing hardware approaches provide the combination of flexibility, efficiency, and minimal design complexity required for practical inter-subarray communication in SRAM-based PIM systems. The need for arbitrary source-destination connectivity at fine granularity, coupled with the requirement for energy-efficient data paths that bypass traditional H-tree routing, remains unaddressed by current solutions.

III. ARCHITECTURE AND DESIGN OF LIBCOM

A. Overview

LIBCom introduces a lightweight switch-based interconnect that enables direct inter-subarray communication within SRAM banks while maintaining compatibility with existing memory designs. As shown in Fig. 2, the key innovation lies in augmenting the traditional H-tree interconnect with a switch network that provides dedicated data paths between subarrays, bypassing the costly round-trip through the bank I/O interface. The switch network operates under the control of peripheral circuitry that manages instruction scheduling

and routing decisions to ensure data consistency and optimal performance.

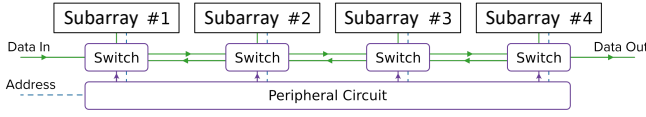


Fig. 2. LIBCom bank architecture with switch-based interconnect

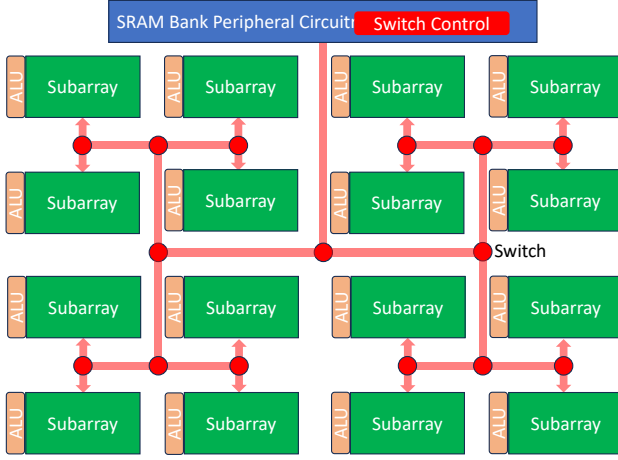


Fig. 3. Bank layout of LIBCom with ALUs

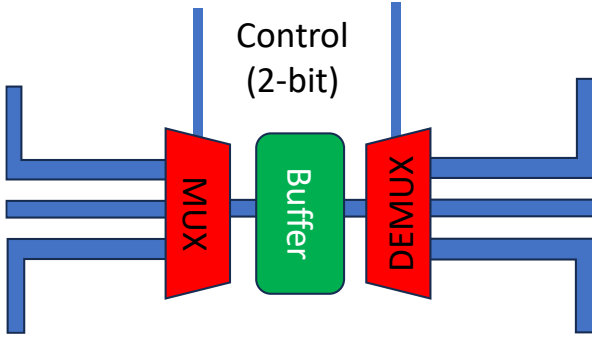


Fig. 4. LIBCom switch design with 2-bit control encoding

B. Switch-Based Interconnect Design

1) *Switch Network Architecture*: The LIBCom switch network consists of lightweight switches positioned throughout the bank following the hierarchical H-tree structure, as illustrated in Fig. 3. Each switch handles point-to-point routing of data between its connected subarray and neighboring switches through the existing H-tree connections. This approach leverages the proven H-tree topology while augmenting it with inter-subarray communication capabilities. The switches are implemented as combinational logic with minimal buffering to maintain the performance characteristics of traditional SRAM access.

Each switch is designed as a simple MUX + Buffer + DEMUX structure controlled by 2-bit signals from the peripheral circuitry, as detailed in Fig. 4. The 2-bit control encoding supports four routing states:

- **00**: Pass-through mode (data flow continues through H-tree, subarray disconnected)
- **01**: Route between one direction and local subarray
- **10**: Route between opposite direction and local subarray
- **11**: Idle state (no active connections)

Burst Transfer Support: To reduce instruction overhead for large data movements, LIBCom supports burst transfers that can move multiple consecutive words in a single operation. The peripheral control circuitry manages burst sequencing by maintaining the same switch routing configuration across multiple clock cycles, while the burst length is encoded in the original Move/Copy instruction. This approach keeps the switch control signals simple (2-bit) while enabling efficient block-level data transfers for matrix operations, vector computations, and other data-intensive PIM algorithms.

Virtual Channel Extension: While the current implementation uses 2-bit switch control for simplicity, the architecture can be extended to support virtual channels for enhanced performance. Virtual channels would enable concurrent inter-subarray transfers between different subarray pairs, eliminating head-of-line blocking and improving throughput for complex PIM workloads with overlapping data movement patterns. This extension would require additional control bits (4-6 total) to encode virtual channel identifiers, flow control information, and arbitration signals. The trade-off between increased control complexity and performance benefits makes virtual channels an attractive option for high-throughput PIM applications, though the current 2-bit design provides a good balance of simplicity and functionality for most use cases.

2) *Switch Control and Operation*: Each switch receives control signals from the peripheral circuitry that specify the source and destination for data routing operations. The control logic determines the appropriate switch configuration to establish the desired data path between subarrays. Since the switch network uses a linear topology, routing between any two subarrays follows a deterministic path through intermediate switches, simplifying the control logic requirements.

To support inter-subarray communication, LIBCom extends the traditional SRAM instruction set with additional commands issued through the I/O interface. The enhanced instruction set includes:

- 1) **Read**: Route data from a specified subarray to the bank output (traditional)
- 2) **Write**: Route data from the bank input to a specified subarray (traditional)
- 3) **Move**: Transfer data from a source subarray to a destination subarray, clearing the source
- 4) **Copy**: Duplicate data from a source subarray to a destination subarray, preserving the source

The Move and Copy instructions are the key additions that enable efficient inter-subarray data transfer without requiring external buffering. These instructions encode the source and destination subarray addresses within the command, allowing the peripheral control circuitry to configure the switch network appropriately. The Compute instruction triggers PE operations while optionally specifying data routing for the results.

C. Peripheral Control System

1) *Instruction Processing and Scheduling*: The peripheral control circuitry manages instruction execution and switch configuration to ensure correct operation and data consistency. Instructions are processed through an in-order queue that can be sized to balance area overhead against robustness requirements. The control system handles instruction scheduling to avoid conflicts and optimize performance through pipelining when possible.

To maintain compatibility with traditional SRAM operation, Read and Write instructions have higher priority than Move and Copy instructions. When conflicts arise, inter-subarray operations (Move/Copy) yield to external memory access requests (Read/Write), ensuring that the primary memory function is not compromised. This priority scheme guarantees that LIBCom can serve as a drop-in replacement for conventional SRAM without affecting system-level memory access patterns.

Read instructions complete in a single clock cycle, while Write, Move, and Copy instructions require two cycles due to the sequential nature of SRAM access operations. However, the second cycle of Write, Move, and Copy instructions can be pipelined with subsequent operations provided they do not create read-after-write hazards to the same subarray.

2) *Memory Consistency and Hazard Detection*: To maintain memory consistency, the control system enforces ordering constraints between instructions that access the same subarray. Write-after-write and read-after-write hazards are detected and resolved through instruction scheduling or the insertion of pipeline bubbles when necessary. The control logic implements a simple scoreboard mechanism to track pending writes to each subarray and prevent conflicting accesses.

For operations requiring deterministic timing, the compiler can assist by reordering instructions to minimize pipeline stalls. For example, a data rotation sequence can be reordered to maximize the overlap between the two-cycle Write/Copy operations and subsequent instructions, reducing the overall execution time.

D. Processing Element Integration

1) *PE Architecture and Placement*: LIBCom integrates lightweight PEs within each subarray to enable in-memory computation (see Fig. 3). Each PE is implemented as a simple ALU capable of performing basic arithmetic, logical, and comparison operations on data stored in the local subarray. Each PE is connected to the subarray's sense amplifiers and write drivers, allowing direct access to memory contents without requiring data movement through the switch network for local computations. The PEs are controlled through the same peripheral circuitry that manages the switch network, providing a unified control interface for both memory operations and computation. This co-design approach minimizes control overhead and enables efficient coordination between data movement and processing operations.

2) *PE Operation and Data Flow*: During computation, each ALU can operate on data within its local subarray while the switch network simultaneously handles inter-subarray data

transfers for other operations. This parallel execution model maximizes throughput by overlapping computation and communication phases. The ALU operations are synchronized with the bank's clock domain and can be pipelined with memory operations to hide computational latency.

Results from ALU computations can be written back to the local subarray or forwarded through the switch network to other subarrays for subsequent processing stages. This flexibility enables complex computational patterns that require data sharing between multiple subarrays while minimizing the energy overhead of data movement.

E. Qualitative Comparison with Traditional SRAM

Table I summarizes the key differences between LIBCom and conventional SRAM architectures across various design metrics and capabilities.

TABLE I
QUALITATIVE COMPARISON: LIBCOM VS. CONVENTIONAL SRAM

Metric	Conventional SRAM	LIBCom
Inter-subarray communication	Via H-tree round-trip	Direct switch paths
Communication latency	2× subarray access + H-tree	1× subarray access
Communication energy	H-tree traversal	Switch control only
Concurrent data flows	Single path (serialized)	Multiple with VC extension
Bulk data transfer	—	Burst support
Data movement flexibility	Fixed H-tree topology	Arbitrary subarray pairs
Network congestion	—	Managed via VC allocation
Memory consistency	Hardware-enforced	Peripheral control + scheduling
Design complexity	Mature, well-understood	Moderate increase
Area overhead	—	Minimal (switches + control)
Power overhead	—	Low (combinational switches)
Scalability	Limited by H-tree	Enhanced with VC support
Compatibility	Standard interface	Drop-in replacement
PIM suitability	Limited by data movement	Optimized for PIM workloads

The comparison highlights LIBCom's key advantages in enabling efficient inter-subarray communication and supporting PIM workloads while maintaining reasonable design complexity and overhead. The direct switch paths eliminate the energy-intensive H-tree traversal required in conventional SRAM for inter-subarray data movement, while the integrated processing elements enable computation without external data movement. The burst transfer capability reduces instruction overhead for large data blocks, and the optional virtual channel extension provides a path toward supporting highly concurrent PIM workloads that require overlapping data movement patterns between multiple subarray pairs.

IV. EVALUATION METHODOLOGY

This section evaluates the LIBCom architecture both on a circuit level and on an architectural level, compared to a traditional SRAM memory.

A. Circuit-Level Evaluations

Both the baseline (a plain SRAM bank) and the one with network interconnects have been simulated using Synopsys Design Compiler using Nangate 45nm technology. The software choice is dictated by its accuracy when simulating custom circuitry. Although Cacti is a more precise simulator when it comes to the baseline, it fails to simulate custom components inside the SRAM. Both the baseline and LIBCom use an SRAM bank as defined in Table II.

TABLE II
BANK PARAMETERS USED FOR BOTH LIBCOM AND THE BASELINE

Bank capacity	#subarrays	Address width	Word size
32 KiB	4	10 bits	32 bits
64 KiB	4	11 bits	32 bits
128 KiB	8	12 bits	32 bits
256 KiB	8	13 bits	32 bits

Both circuits have been evaluated for different instructions:

- 1) Read: random data is written to random addresses. The circuit is evaluated when reading those addresses
- 2) Write: the circuit is evaluated when writing random data to random addresses
- 3) Copy: random data is written to random addresses. The circuit is evaluated when copying that data to other random addresses
- 4) Move: random data is written to random addresses. The circuit is evaluated when moving that data to other random addresses

The evaluated metric is an average over 5000 random iterations. While the baseline has no ideal instruction ordering, the evaluated circuit is sensitive to instruction ordering. Instructions have hence been ordered in an ideal manner to avoid reading from a subarray the cycle following a write to that subarray, whenever possible. A hardware queue capable of holding 2 instructions is used to cope with scenarios when instruction re-ordering is insufficient.

B. Architectural-Level Evaluations

To evaluate LIBCom's effectiveness for real PIM workloads, we conducted system-level simulations using a modified version of MultiPIM [23], a cycle-accurate simulator for processing-in-memory architectures. We extended MultiPIM to incorporate our switch-based interconnect model with realistic latency and energy parameters derived from the circuit-level analysis presented in Section IV-A.

Simulation Configuration: We configured the modified MultiPIM to model LIBCom banks with the specifications outlined in Table II. The baseline configuration uses traditional SRAM banks with H-tree interconnects, while the LIBCom configuration enables direct inter-subarray data movement

through the switch network. The memory latency and energy models are based on our circuit evaluations to ensure accurate representation of LIBCom's performance characteristics.

Workload Selection: We selected two representative applications that exhibit significant inter-subarray communication requirements: General Matrix Multiplication (GEMM) and Breadth-First Search (BFS).

Metrics: The architectural evaluation focuses on overall application performance (execution time), energy consumption, and network utilization patterns. We measure the effectiveness of LIBCom's inter-subarray communication capabilities compared to traditional H-tree routing for these representative PIM workloads.

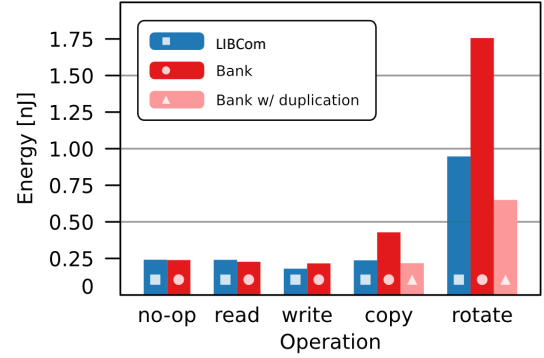


Fig. 5. Energy consumption of various instructions

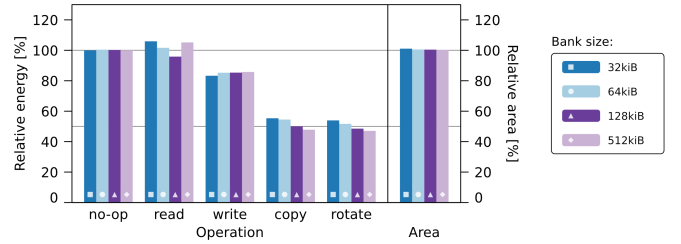


Fig. 6. Relative area overhead and energy consumption per bank size

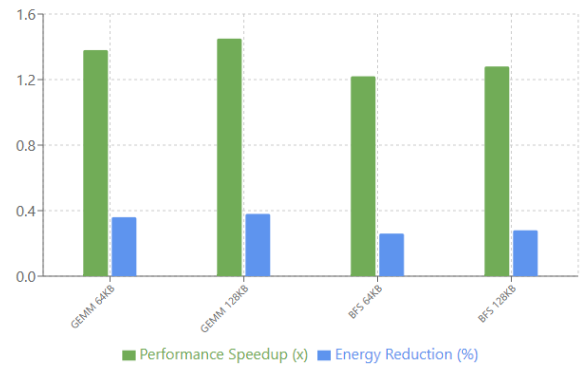


Fig. 7. LIBCom evaluation results across application workloads showing performance speedup and energy reduction

V. EVALUATION RESULTS AND DISCUSSIONS

A. Circuit-Level Performance and Energy Analysis

This section compares the LIBCom memory to a traditional SRAM memory when executing the instructions defined in Section IV-A.

1) *Operating Frequency and Cycle Time*: Both the baseline and LIBCom have the same worst-case data arrival time. Hence the baseline and LIBCom have been simulated with a clock frequency of 500MHz. Therefore, differences in timing between the two circuits come down to the number of cycles they require to perform each instruction. LIBCom enables copy and move operations to complete in a single cycle through ideal instruction ordering and pipelining, while traditional SRAM requires two cycles for these operations.

2) *Circuit-Level Energy and Area Results*: The energy consumption of both circuits is evaluated for the instructions described in Section IV-A. Fig. 5 shows the energy consumption for various instructions, while Fig. 6 presents the relative energy consumption and area overhead compared to the baseline across different bank sizes.

Although LIBCom requires slightly more energy for simple read operations (about +5%), significant energy gains can be seen for copy and move operations (-45% and -47% respectively). This is mainly due to the lower number of cycles LIBCom requires compared to the baseline. While LIBCom can perform a copy operation in one clock cycle through pipelining, the baseline requires one cycle for the read and one for the write operation without pipelining capability.

The area overhead introduced by LIBCom remains below 5% across all evaluated configurations. As bank size increases, the benefits for copy and move operations grow because LIBCom's overhead slightly increases with the number of subarrays, but the energy required for memory operations increases with subarray size. LIBCom hence brings growing benefits as memory size increases.

B. Application-Level Performance and Energy Results

Our architectural-level evaluation demonstrates significant improvements for both GEMM and BFS workloads that require inter-subarray communication. Fig. 7 presents comprehensive results showing performance speedup and energy reduction for LIBCom across different workload configurations.

Performance Gains: Matrix multiplication (GEMM) shows substantial improvement with 38-45% speedup across different matrix sizes, benefiting significantly from LIBCom's burst transfer capabilities when moving matrix blocks between subarrays. Graph traversal (BFS) achieves 22-28% performance improvement despite irregular access patterns, with reduced latency for inter-subarray data movement when frontier vertices are distributed across multiple subarrays.

Energy Efficiency: LIBCom achieves 26-38% energy reduction for both workloads, derived from the circuit-level analysis showing 45-47% energy savings for copy and move operations. GEMM demonstrates 36-38% energy reduction due to regular data movement patterns that efficiently utilize burst transfers, while BFS shows 26-28% energy improvement despite less predictable access patterns. The energy savings result from eliminating energy-intensive H-tree traversal and reducing cycle counts for inter-subarray data transfers.

C. Further Discussions

Manufacturing Considerations: Because LIBCom requires changes to the SRAM H-tree, it is bound to pose some manufacturing challenges. However, we believe that such challenges will have to be addressed. For this reason, currently proposed solutions all include changes to the SRAM architecture, as explained in Section II. This work should nonetheless prove an easier challenge as it only requires modifications to the interconnect, and no modification is required to the subarray itself.

Applicability to Other Memory Technologies: The switch-based interconnect concept could be applied to other memory technologies, though implementations would require substantial modifications. For DRAM banks, the fundamentally different access patterns and refresh requirements would complicate control logic significantly, despite DRAM's longer access latencies potentially benefiting more from concurrent data transfers. Non-volatile memories like ReRAM, PCM, and STT-MRAM could also benefit from similar interconnect enhancements, as these technologies often suffer from higher write latencies and limited endurance, making efficient data movement even more critical. The switch-based approach could reduce unnecessary write operations by enabling direct data transfers without intermediate buffering, though technology-specific constraints such as asymmetric read/write latencies and wear-leveling considerations would need to be addressed.

VI. CONCLUSIONS

This paper presents LIBCom, a lightweight switch-based interconnect that enables efficient inter-subarray communication in SRAM-based processing-in-memory systems. By augmenting traditional H-tree interconnects with direct switch paths between subarrays, LIBCom eliminates the costly round-trip data movement that limits PIM performance. Our comprehensive evaluation demonstrates significant benefits across multiple levels. At the circuit level, LIBCom achieves 45-47% energy reduction for copy and move operations while maintaining minimal area overhead below 5%. At the application level, representative PIM workloads show 22-45% performance improvement and 26-38% energy reduction compared to traditional SRAM architectures. The lightweight nature of LIBCom, requiring only peripheral control modifications without changes to subarray internals, makes it practical for integration into existing memory designs. Furthermore, the core principles of switch-based interconnection can potentially be adapted to other memory technologies, suggesting broader applicability beyond SRAM-based systems. These characteristics position LIBCom as an attractive solution for next-generation processing-in-memory architectures where efficient inter-subarray communication is critical for computational performance.

REFERENCES

- [1] W. A. Wulf and S. A. McKee, "Hitting the memory wall: Implications of the obvious," *ACM SIGARCH computer architecture news*, vol. 23, no. 1, pp. 20–24, 1995.
- [2] S. Kanev, J. P. Darago, K. Hazelwood, P. Ranganathan, T. Moseley, G.-Y. Wei, and D. Brooks, "Profiling a warehouse-scale computer," *ACM SIGARCH Computer Architecture News*, vol. 43, no. 3, pp. 158–169, 2015.
- [3] K. Asifuzzaman, N. R. Miniskar, A. R. Young, F. Liu, and J. S. Vetter, "A survey on processing-in-memory techniques: Advances and challenges," *Memories-Materials, Devices, Circuits and Systems*, vol. 4, p. 100022, 2022.
- [4] S. Ghose, A. Boroumand, J. S. Kim, J. Gomez-Luna, and O. Mutlu, "Processing-in-memory: A workload-driven perspective," *IBM Journal of Research and Development*, vol. 63, no. 6, pp. 3–1, 2019.
- [5] M. Fernandez, J. Gomez-Luna, S. Ghose, A. Boroumand, and O. Mutlu, "A survey of resource management for processing-in-memory and near-memory processing architectures," *ACM Computing Surveys*, vol. 53, no. 4, pp. 1–38, 2020.
- [6] H. Son, G. Jonatan, W. Xiangyu, H. Cho, K. Shivdikan, J. L. Abellán, A. Joshi, D. Kaeli, and J. Kim, "PIMnet: A Domain-Specific Network for Efficient Collective Communication in Scalable PIM," in *Proceedings of the 31st IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1543–1557.
- [7] M. Gao, J. Pu, X. Yang, M. Horowitz, and C. Kozyrakis, "Tetris: Scalable and efficient neural network acceleration with 3d memory," vol. 45, no. 1, p. 751–764, apr 2017.
- [8] S. Venkataramani, A. Ranjan, S. Banerjee, D. Das, S. Avancha, A. Jagannathan, A. Durg, D. Nagaraj, B. Kaul, P. Dubey, and A. Raghunathan, "Scaleddeep: A scalable compute architecture for learning and evaluating deep networks," in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 13–26.
- [9] S. Venkataramani, V. Srinivasan, W. Wang, S. Sen, J. Zhang, A. Agrawal, M. Kar, S. Jain, A. Mannari, H. Tran, Y. Li, E. Ogawa, K. Ishizaki, H. Inoue, M. Schaal, M. Serrano, J. Choi, X. Sun, N. Wang, C.-Y. Chen, A. Allain, J. Bonano, N. Cao, R. Casatuta, M. Cohen, B. Fleischer, M. Guillorn, H. Haynie, J. Jung, M. Kang, K.-h. Kim, S. Koswatta, S. Lee, M. Lutz, S. Mueller, J. Oh, A. Ranjan, Z. Ren, S. Rider, K. Schelm, M. Scheuermann, J. Silberman, J. Yang, V. Zalani, X. Zhang, C. Zhou, M. Ziegler, V. Shah, M. Ohara, P.-F. Lu, B. Curran, S. Shukla, L. Chang, and K. Gopalakrishnan, "Rapid: Ai accelerator for ultra-low precision training and inference," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 153–166.
- [10] M. Gao, X. Yang, J. Pu, M. Horowitz, and C. Kozyrakis, "Tangram: Optimized coarse-grained dataflow for scalable nn accelerators," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19, 2019, p. 807–820.
- [11] L. Sonnino, S. Shresthamali, Y. He, and M. Kondo, "Daism: Digital approximate in-sram multiplier-based accelerator for dnn training and inference," 2024.
- [12] C. Eckert, X. Wang, J. Wang, A. Subramaniyan, R. Iyer, D. Sylvester, D. Blaauw, and R. Das, "Neural cache: Bit-serial in-cache acceleration of deep neural networks," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018, pp. 383–396.
- [13] H. Jin *et al.*, "ReHy: a ReRAM-based digital/analog hybrid PIM architecture for accelerating CNN training," *IEEE TPDS*, vol. 33, no. 11, pp. 2872–2884, Dec 2022.
- [14] S. Aga, S. Jeloka, A. Subramaniyan, S. Narayanasamy, D. Blaauw, and R. Das, "Compute caches," in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 481–492.
- [15] V. Seshadri, Y. Kim, C. Fallin, D. Lee, R. Ausavarungnirun, G. Pekhimenko, Y. Luo, O. Mutlu, P. B. Gibbons, M. A. Kozuch, and T. C. Mowry, "Rowclone: Fast and energy-efficient in-dram bulk data copy and initialization," in *2013 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2013, pp. 185–197.
- [16] Z. Zhou, C. Li, F. Yang, and G. Sun, "Dimm-link: Enabling efficient inter-dimm communication for near-memory processing," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 302–316.
- [17] W. Sun, Z. Li, S. Yin, S. Wei, and L. Liu, "Abc-dimm: Alleviating the bottleneck of communication in dimm-based near-memory processing with inter-dimm broadcast," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 237–250.
- [18] M. Izhar, S. Ahmad, and S. A. Rahman, "Logic circuit implementation for enabling sram based in memory computing," in *2022 5th International Conference on Multimedia, Signal Processing and Communication Technologies (IMPACT)*, 2022, pp. 1–5.
- [19] Z. Lin, Z. Tong, F. Wang, J. Zhang, Y. Zhao, P. Sun, T. Xu, C. Zhang, X. Li, X. Wu, W. Lu, C. Peng, Q. Zhao, and J. Chen, "In situ storing 8t sram-cim macro for full-array boolean logic and copy operations," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 5, pp. 1472–1486, 2023.
- [20] S. Okumura, S. Yoshimoto, K. Yamaguchi, Y. Nakata, H. Kawaguchi, and M. Yoshimoto, "7t sram enabling low-energy simultaneous block copy," in *IEEE Custom Integrated Circuits Conference 2010*, 2010, pp. 1–4.
- [21] G. Matsukawa, Y. Nakata, Y. Kimi, Y. Sugure, M. Shimozawa, S. Oho, H. Kawaguchi, and M. Yoshimoto, "A low-latency dmr architecture with efficient recovering scheme exploiting simultaneously copiable sram," in *ARCS 2014; 2014 Workshop Proceedings on Architecture of Computing Systems*, 2014, pp. 1–5.
- [22] F. Duarte and S. Wong, "Cache-based memory copy hardware accelerator for multicore systems," *IEEE Transactions on Computers*, vol. 59, no. 11, pp. 1494–1507, 2010.
- [23] C. Yu, S. Liu, and S. Khan, "Multipim: A detailed and configurable multi-stack processing-in-memory simulator," *Computer Architecture Letters*, vol. 20, no. 2, pp. 54–57, 2021.